

Lab 4: Loops

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** while, do-while, for, nested loops, break/continue

Objectives

- Choose the correct loop form for the problem (counter-controlled vs. condition-controlled vs. at-least-once).
- Build nested loops to print patterns and search a 2D space.
- Use **break** and **continue** to control loop flow.
- Translate a mathematical sum or algorithm into an iterative program.

Quick Reference

Three loop forms.

```
while (cond) { /* tested first */ }
do { /* runs at least once */ } while (cond);
for (init; cond; update) { /* counter-controlled */ }
```

Loop control. **break** exits the nearest loop; **continue** skips to the next iteration of the nearest loop.

Common pattern: accumulator.

```
int sum = 0;
for (int i = 1; i <= n; i++) sum += i;
```

Common pattern: flag + early exit.

```
int found = 0;
for (int i = 2; i*i <= n; i++)
    if (n % i == 0) { found = 1; break; }
```

Quick Experiments (Try It)

Try It 1: Off-by-one: < versus <=

Run both versions and record each result:

```
#include <stdio.h>
int main(void) {
    int sum = 0;
    for (int i = 1; i < 5; i++) sum += i;    /* version A */
    printf("< 5: %d\n", sum);

    sum = 0;
    for (int i = 1; i <= 5; i++) sum += i;  /* version B */
    printf("<= 5: %d\n", sum);
    return 0;
}
```

1. What are the two sums? Which version sums 1 through 5 inclusive?
2. Which condition gives you the n -th triangular number $T_n = n(n+1)/2$?
3. Off-by-one errors are among the most common bugs. Describe a quick mental check you can use: “Does my loop body run once for $i = 1$ and once for $i = n$?”

Try It 2: break vs. continue – same condition, different outcomes

Start with this skeleton:

```
#include <stdio.h>
int main(void) {
    for (int i = 1; i <= 10; i++) {
        if (i % 3 == 0) { /* ??? */ }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

1. Replace `/* ??? */` with `continue`; Run it. Which numbers appear?
2. Replace `continue` with `break`; Run it. Which numbers appear now?
3. Explain in one sentence the difference between what the two keywords do.

Try It 3: Semicolon after while – the empty-body loop

Run and observe carefully:

```
#include <stdio.h>
int main(void) {
    int n = 5;
    while (n > 0); /* <-- notice the semicolon */
    printf("n = %d\n", n--);
    return 0;
}
```

What happens? (You may need to force-stop the program.) Remove the semicolon and run again. What is the correct output? This is the same bug as in the take-home section – recognizing it by sight is a valuable skill.

In-Lab Exercises (target: 2 hours)**In-Lab Exercise 1: Guess the Number (25 min)**

Choose a secret number (e.g. 42) in your code. Repeatedly read a guess from the user with a `do-while` loop, printing `Too high`, `Too low`, or `Correct` as appropriate. After the user wins, print the number of guesses they used.

In-Lab Exercise 2: Prime Check with Early Exit (25 min)

Read a positive integer n and decide whether it is prime. Use a `for` loop from $i = 2$ to $\sqrt{n}$ and `break` as soon as a divisor is found. Handle $n \leq 1$ and $n = 2$ as special cases.

In-Lab Exercise 3: Pyramid Pattern (35 min)

Read an integer N and print a centered pyramid of asterisks with N rows. For $N = 5$:

```
  *
 ***
*****
*****
*****
```

Hint: row r (1-indexed) has $N - r$ leading spaces and $2r - 1$ asterisks.

In-Lab Exercise 4: Approximate e^x via Taylor Series (35 min)

Read x as a `double` and approximate $e^x = \sum_{n=0}^{\infty} x^n/n!$. Stop when the magnitude of the current term is below 10^{-6} . **Hint:** compute each next term incrementally as `term *= x / n`, where `n` is the new term index. Test with $x = 0, 1, 2$ and compare against `exp(x)` (link with `-lm`).

AI Collaboration

Try these prompts with an AI tool:

- “Explain the off-by-one error: when should a `for` loop use `i < n` versus `i <= n`?”
- “My `while` loop runs forever. Walk me through a checklist for finding infinite-loop bugs.”
- “Rewrite this `for` loop as a `while` loop and explain when each form is preferable: `for (int i = 0; i < n; i++) { ... }`”

Critical check – verify, don’t just accept: Ask the AI to write a loop that sums integers from 1 to N inclusive. Before running, trace the AI’s loop by hand for $N = 5$, writing the accumulator value after each iteration. Then run the program for $N = 1$, $N = 5$, and $N = 0$. Compare the actual output to your trace and to the AI’s stated result.

Know the limit: AI tools often generate loops with subtle off-by-one errors in the boundary condition – especially when converting between 0-indexed and 1-indexed thinking – and rarely flag this without being explicitly asked.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Off-by-one

What is the printed output?

```
#include <stdio.h>
int main(void) {
    int sum = 0;
    for (int i = 1; i < 5; i++) sum += i;
    printf("%d\n", sum);
    return 0;
}
```

Trace & Predict 2: Nested loop with continue

Predict the exact output:

```
#include <stdio.h>
int main(void) {
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
```

```
        if (i == j) continue;
        printf("(%d,%d) ", i, j);
    }
    printf("\n");
}
return 0;
}
```

Find the Bug 1: Infinite loop

Why does this program never terminate? Fix it (two reasonable fixes exist; show one).

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    while (n > 0);
    {
        printf("%d\n", n);
        n--;
    }
    return 0;
}
```

Write Code on Paper 1: Sum and average of N inputs

Write a complete program that reads N followed by N integers, then prints their sum and their average (as a double with two decimal places).

Write Code on Paper 2: Multiplication table

Write a complete program that reads an integer N and prints the $N \times N$ multiplication table. Each cell must use a field width of 4 so the columns align.